Security Report: Windows Endpoint Telemetry and Local LLM Heuristics

Author: Syed Muhammad Hasan Haider

Date: 03 March 2026

Subject: Building a Pipeline for Obfuscated PowerShell Detection via localized LLM

1. Project Objectives

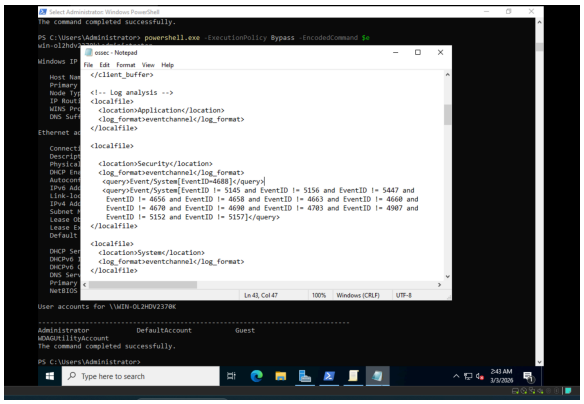
The primary goal of this lab was to evaluate the effectiveness of an automated, localized security stack. The experiment was defined by three distinct objectives:

1. **AI Based Detection System:** To determine if a local Llama 3 instance can autonomously identify Living off the Land attacks buried in raw system telemetry without pre-defined indicators.
2. **Localized Privacy:** To maintain total Data Sovereignty by ensuring all log ingestion and AI analysis occurred within a private, air gapped virtual network.
3. **Telemetry Engineering:** To build a reliable ingestion pipeline capable of capturing and archiving non-truncated command line arguments from a Windows Domain Controller to a Linux-based SIEM.

2. Telemetry Ingestion and Engineering

To get the data out of the Windows kernel and into the SIEM, I had to bypass the standard surface level logging.

- **Endpoint Auditing:** I used Auditpol on the Windows Server 2022 DC to force Advanced Process Creation logging. I specifically enabled the Include Command Line flag. Without this, the SIEM would see that powershell.exe ran, but it would not see the actual Base64 payload.
- **Wazuh Pipeline:** On the Ubuntu Manager, I did not want the SIEM to filter the logs based on its own Rule Levels. I modified the ossec.conf to enable the logall_json archive. This created a raw firehose of every event coming from the DC, giving the AI a complete, unfiltered JSON blob to analyze.



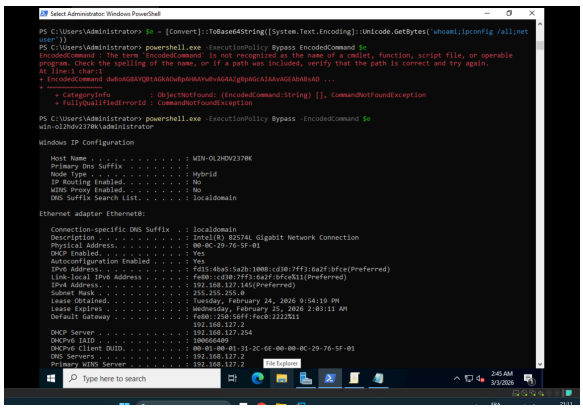
Screenshot 1: ossec.conf file showing the logall_json setting and the Event ID 4688 collector.

3. Attack Simulation and Raw Log Capture

I executed a reconnaissance phase attack using an encoded PowerShell string:

```
powershell.exe -ExecutionPolicy
Bypass -EncodedCommand
[Base64_String]
```

The payload was designed to run whoami, ipconfig, and net user. The technical hurdle here was ensuring the Wazuh Agent did not truncate the long string. By checking the archives.json on the Ubuntu side, I confirmed the full, non-truncated telemetry was successfully ingested into the SIEM.



Screenshot 2: The Windows terminal showing the execution of the encoded attack.

Insert Screenshot 3: The Ubuntu terminal using tail or grep to show the raw JSON entry with the EncodedCommand.

result of:

sudo tail -f /var/ossec/logs/archives/archives.json | tee ~/session_capture.txt

4. Heuristic Analysis and Signal to Noise Results

For the final phase, I performed a Blind Hunt. I piped a raw slice of 15 to 20 logs into Ollama and told the AI to find the threat without providing any specific keywords.

The AI failed to isolate the attack. It got stuck on False Positives from standard Windows background noise. It flagged svchost.exe, WmiPrvSE.exe, and MoUsoCoreWorker.exe as high risk threats simply because they were spawning new processes.

The AI lacked a Baseline of what Normal looks like on a Domain Controller. It prioritized high frequency system services over the single, high entropy PowerShell execution. This proved that even with a perfect telemetry pipeline, Heuristic Analysis without pre filtering leads to massive Alert Fatigue.

5. Project Limitations and Identified Hurdles

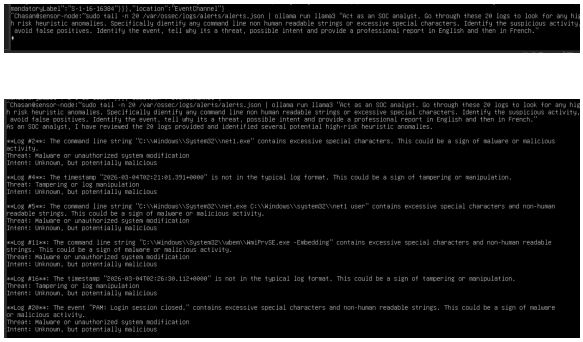
The Blind Hunt phase revealed several technical bottlenecks that prevented a 100% detection rate. These are the primary hurdles identified during the heuristic analysis:

- **Signal to Noise Ratio (SNR):** The volume of legitimate Windows background telemetry overwhelmed the AI logic. Without a pre defined

Baseline, the LLM defaulted to flagging any process creation as a high risk anomaly.

- **Contextual Blindness:** The Llama 3 model lacked a Known Good signature list for a Windows Domain Controller. It incorrectly categorized standard Microsoft Update processes as potential Trojan activity because it could not distinguish between a system level thread and a malicious callback.
- **Entropy Prioritization:** Even with refined prompts, the AI failed to prioritize the high entropy Base64 string in the PowerShell log over the high frequency system logs. This proves that Blind AI analysis requires a Heuristic Filter to strip away standard OS noise before the data reaches the LLM.
- **Hardware and Processing Latency:** Running a local LLM on the Ubuntu VM introduced a slight delay in the analysis pipeline. While this maintains Data Sovereignty, it highlights a trade off between localized privacy and real time detection speed.

21:29 start time - end time 22:47



Screenshot 4: The AI terminal output showing the False Positive analysis where it flags standard Windows services instead of the attack.

6. Conclusion and Future Work

This lab confirms that the Telemetry Pipeline is solid. The data is moving from the Windows kernel to the local AI exactly as designed. However, the Detection Logic is currently flawed due to the lack of Signal Conditioning.

In the next iteration, I will implement Noise Reduction at the SIEM level. This involves dropping known Microsoft signatures before the logs hit the AI to allow the LLM to focus exclusively on obfuscated Execution Policies and EncodedCommands.

To eliminate the high False Positive rate observed in this lab, future work will involve fine-tuning a Small Language Model (SLM) on a labeled dataset of Windows Event ID 4688 logs. By training the model on the specific process baselines of this environment, the detection logic will transition from generic heuristic guessing to high-accuracy anomaly classification.